计算机组成与结构
——运算方法（乘除法专题巩固）

李瑞　副教授
蒋志平　讲师
计算机科学与技术学院

乘法的“表格式计算”

“表格式计算”—乘法

- 以原码一位乘法为例：竖式是直观的，步骤步骤累加是可理解的，表格式是一眼看上去懵逼的...

例，[X]原=0.1101, [Y]原=1.1011，求积

解： 符号，$S = 0 \oplus 1 = 1$； 数值部分如下：

模拟手工乘法

```
          1 1 0 1
x         1 0 1 1
-------------------
          1 1 0 1
        1 1 0 1
      0 0 0 0
    1 1 0 1
-------------------
1 0 0 0 1 1 1 1
```

通过部分积D累加逐层结果

第一层 加1101	0 0 0 0 + 1 1 0 1 1 1 0 1	<-部分积D，初值为0 D = D + X
第二层 加1101	0 1 1 0 1 + 1 1 0 1 0 1 0 0 1 1 1	D右移 D = D>>1 + X
第三层 加0000	1 0 0 1 1 1 + 0 0 0 0 0 0 1 0 0 1 1 1	D右移 D = D>>1 + X
第四层 加1101	0 1 0 0 1 1 1 + 1 1 0 1 0 0 0 1 0 0 0 1 1 1 1	D右移 D = D>>1 + X

<<是左移操作， >>是右移操作

“表格式计算”—乘法

- 以原码一位乘法为例：竖式是直观的，步骤步骤累加是可理解的，表格式是一眼看上去懵逼的...

通过部分积D累加逐层结果

第一层 加1101	0 0 0 0 + 1 1 0 1 1 1 0 1	<-部分积D，初值为0 D = D + X
第二层 加1101	0 1 1 0 1 + 1 1 0 1 0 1 0 0 1 1 1	D右移 D = D>>1 + X
第三层 加0000	1 0 0 1 1 1 + 0 0 0 0 0 0 1 0 0 1 1 1	D右移 D = D>>1 + X
第四层 加1101	0 1 0 0 1 1 1 + 1 1 0 1 0 0 0 1 0 0 0 1 1 1 1	D右移 D = D>>1 + X

<<是左移操作， >>是右移操作

部分积D的符号（建议至少2位）

部分积D

粗线分隔D和Y

乘数Y

Y总是顶格对齐，D总右移，等效于竖式中X左移

2位符号，防止溢出时右移出现负数

符号	D	Y	Y_0	操作说明
0 0	0 0 0 0	1 0 1	1	D/Y初始化
0 0	1 1 0 1	1 0 1	1	+[X]原
0 0	1 1 0 1	1 0 1	1	
0 0	0 1 1 0 1	1 0	1	D/Y右移1位
0 0	1 1 0 1 0	1 0	1	+[X]原
0 1	0 0 1 1 1	1 0	1	
0 0	1 0 0 1 1 1	1	0	D/Y右移1位
0 0	0 0 0 0	1	0	+0
0 0	1 0 0 1 1 1	1	0	
0 0	0 1 0 0 1 1 1		1	D/Y右移1位
0 0	1 1 0 1		1	+[X]原
0 1	0 0 0 1 1 1 1		1	
0 0	1 0 0 0 1 1 1 1		1	D/Y右移1位

乘数用尽，计算结束

“表格式计算”—乘法

练习，[X]原=0.100111, [Y]原=1.100111，求积

1. 设一临时变量$D=0$
2. 若Y末位为1，$D=D+X$
3. D右移一位；Y右移一位
4. 若Y有剩余位数，回到第2步
5. 结束

符号	D	Y	Y_0	操作说明
0 0	0 0 0 0 0 0	1 0 0 1 1	1	

“表格式计算”—乘法

练习，[X]原=0.100111, [Y]原=1.100111，求积

1. 设一临时变量$D=0$
2. 若Y末位为1，$D=D+X$
3. D右移一位；Y右移一位
4. 若Y有剩余位数，回到第2步
5. 结束

符号	D	Y	Y_0	操作说明
0 0	0 0 0 0 0 0	1 0 0 1 1	1	D/Y初始化
0 0	1 0 0 1 1 1			+[X]原
0 0	1 0 0 1 1 1			
0 0	0 1 0 0 1 1 1	1 0 0 1	1	D/Y右移1位
0 0	1 0 0 1 1 1 0			+[X]原
0 0	1 1 1 0 1 0 1	1 0 0 1	1	
0 0	0 1 1 1 0 1 0 1	1 0 0 1	1	D/Y右移1位
0 0	1 0 0 1 1 0 0	1 0 0 1	1	+[X]原
0 1	0 0 0 1 0 0 0 1	1 0 0 1	1	
0 0	1 0 0 0 1 0 0 0 1	1 0 0	1	D/Y右移1位
				+0
0 0	0 1 0 0 0 1 0 0 0 1	1 0	1	D/Y右移1位
				+0
0 0	0 0 1 0 0 0 1 0 0 0 1		1	D/Y右移1位
0 0	1 0 0 1 1 1			+[X]原
0 0	1 0 1 1 1 1 0 0 0 1		1	
0 0	0 1 0 1 1 1 1 0 0 0 1			D/Y右移1位

原码两位乘法

原码二位乘法

原码二位乘法，

一次乘两位数字？这个在十进制下很反人类啊…，但在二进制下，似乎还好…

```
          1 1 0 1
        × 1 0 0 1
        ---------
          1 1 0 1        +|X|
        1 1 0 1 0        +2|X|
        ---------
        0 1 1 1 0 1 0 1
```

```
          1 1 0 1
        × 1 1 0 0
        ---------
          0 0 0 0        +0
        1 0 0 1 1 1      +3|X|
        ---------
        1 0 0 1 1 1 0 0
```

规则似乎很直观：

$$\begin{cases} Y_{i+1}Yi = 00, +0 \\ Y_{i+1}Yi = 01, +|X| \\ Y_{i+1}Yi = 10, +2|X| \\ Y_{i+1}Yi = 11, +3|X| \end{cases}$$

规则还可以更进一步优化：

$3|X| = +4|X| - |X|$

⇒先减$|X|$，平移2位后加$|X|$

怎么做减法？把X和D用补码表示

```
              0 0 1 1 0 1
            × 0 0 1 1 0 0
            -------------
              0 0 0 0 0 0
          1 1 0 0 1 1      −|X|
        0 0 1 1 0 1        +|X|
        -------------
        0 1 1 0 0 1 1 1 0 0
```

原码二位乘法

规则还可以更进一步优化：

$$3|X| = +4|X| - |X|$$

⇒先减$|X|$，平移2位后加$|X|$

怎么做减法？把X和D用补码表示

然而，还有一些复杂情况，例如…

```
          0 0 1 1 0 1
    ×     0 0 1 1 0 0
  ---------------------
          0 0 0 0 0 0
        1 1 0 0 1 1
      0 0 1 1 0 1
  ---------------------
    0 0 1 0 0 1 1 1 0 0
```

```
          0 0 1 1 0 1
    ×     0 0 1 0 1 1
  ---------------------
          1 1 0 0 1 1
        ? ? ? ? ? ?
      ? ? ? ? ? ?
    … … … … … …
  ---------------------
    ? ? ? ? ? ? ? ? 1 1
```

```
          0 0 1 1 0 1
    ×     0 0 1 1 1 1
  ---------------------
          1 1 0 0 1 1
        ? ? ? ? ? ?
      ? ? ? ? ? ?
    … … … … … …
  ---------------------
    ? ? ? ? ? ? ? ? 1 1
```

原码二位乘法

规则还可以更进一步优化：

$3|X| = +4|X| - |X|$

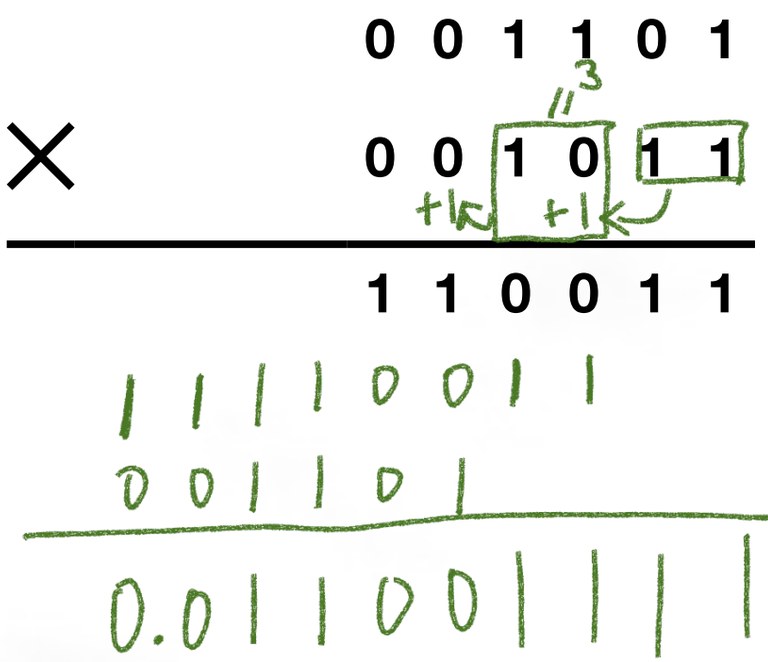
⇒先减$|X|$，平移2位后加$|X|$

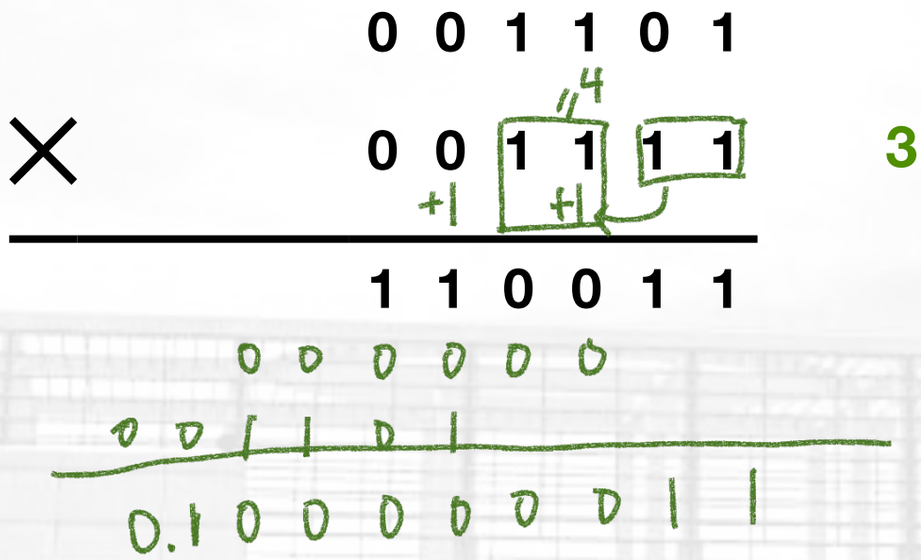
怎么做减法？把X和D用补码表示

好在这些复杂情况，还是使用相同的方法 + 进位机制！

```
      0 0 1 1 0 1
×     0 0 1 1 0 0
-------------------
      0 0 0 0 0 0
    1 1 0 0 1 1
  0 0 1 1 0 1
-------------------
  0 0 1 0 0 1 1 1 0 0
```

```
      0 0 1 1 0 1
×     0 0 1 0 1 1
-------------------
      1 1 0 0 1 1
    0 0 1 1 0 1
  0 0 1 1 0 1
-------------------
  0 0 1 0 0 1 1 1 0 0
```





补充的规则：

1. 遇到>=3倍情况时，平移2位后加$|X|$，相当于给下一组乘数进位+1；
2. 当=4倍情况时，不减$|X|$直接平移2位，并进位
3. 使用一个标志C指示下一组需要加上进位

原码二位乘法

带大家练习一哈，[X]原=0.100111, [Y]原=1.100111，求积

解：[2X]补=01.001110; [-X]补=11.011001

符号			D						Y				Y_1	Y_0	操作说明
0	0	0	0	0	0	0	0	0	1	0	0	1	1	1	

原码二位乘法

带大家练习一哈，[X]原=0.100111, [Y]原=1.100111，求积

解：[2X]补=01.001110; [-X]补=11.011001

3位符号防溢出　部分积D　粗线分隔D和Y　乘数Y　关注后两位

符号			D						Y				Y_1	Y_0	操作说明
0	0	0	0	0	0	0	0	0	1	0	0	1	1	1	D/Y初始化
1	1	1	0	1	1	0	0	1	1	0	0	1	1	1	尾数11，前无进位，-X，进位
1	1	1	0	1	1	0	0	1	1	0	0	1	1	1	
1	1	1	1	1	0	1	1	0	0	1	1	0	0	1	D/Y右移2位
0	0	1	0	0	1	1	1	0	0	0	1	0	0	1	尾数01，前有进位，+2X，不进位
0	0	1	0	0	0	1	0	0	0	1	1	0	0	1	
0	0	0	0	1	0	0	0	1	0	0	0	1	1	0	D/Y右移1位
0	0	1	0	0	1	1	1	0	0	0	0	0	1	0	尾数10，前无进位，+2X，不进位
0	0	1	0	1	1	1	1	1	0	0	0	1	1	0	
0	0	0	0	1	0	1	1	1	1	1	0	0	0	1	D/Y右移2位

负数右移仍为负

乘数用尽，计算结束

原码二位乘法

再来练习一哈，[X]原=0.100111, [Y]原=1.101110，求积

解：[2X]补=01.001110; [-X]补=11.011001

符号			D						Y				Y_1	Y_0	操作说明
0	0	0	0	0	0	0	0	0	1	0	1	1	1	0	D/Y初始化

原码二位乘法

再来练习一哈，[X]原=0.100111, [Y]原=1.101110，求积

解：[2X]补=01.001110; [-X]补=11.011001

3位符号防溢出

部分积D

粗线分隔D和Y

乘数Y

关注后两位

符号			D						Y				Y_1	Y_0	操作说明
0	0	0	0	0	0	0	0	0	1	0	1	1	1	0	D/Y初始化
0	0	1	0	0	1	1	1	0	1	0	1	1	1	0	尾数10，前无进位，+2X，不进位
0	0	1	0	0	1	1	1	0	1	0	1	1	1	0	
0	0	0	0	1	0	0	1	1	1	0	1	0	1	1	D/Y右移2位
1	1	1	0	1	1	0	0	1	0	0	1	0	1	1	尾数11，前无进位，-X，进位
1	1	1	1	0	1	1	0	0	1	0	1	0	1	1	
1	1	1	1	1	1	0	1	1	0	0	1	0	1	0	D/Y右移2位
1	1	1	0	1	1	0	0	1	0	0	0	0	1	0	尾数10，前有进位，-X，进位
1	1	1	0	1	0	1	0	0	0	0	1	0	1	0	
1	1	1	1	1	0	1	0	1	0	0	0	0	1	0	D/Y右移2位
0	0	0	1	0	0	1	1	1	0	0	0	0	0	0	尾数00，前有进位，+X
0	0	0	0	1	1	1	0	0	0	0	0	0	1	0	

负数右移仍为负

乘数用尽，计算结束

乘数用尽后仍有进位，继续加

Booth乘法（补码乘法）

Booth乘法（补码乘法）

```
          0 0 1 1 0 1
×         0 0 1 1 1 0
-----------------------
          0 0 0 0 0 0
        0 0 1 1 0 1
      0 0 1 1 0 1
    0 0 1 1 0 1
  0 0 0 0 0 0
-----------------------
0 0 0 1 0 1 1 0 1 1 0
```

<—无脑计算需要6步

v.s

略用心机只需3步—>

能不能更简化！？

先看一个日常使用的技巧：

$$12345 \times 1001 = 12345 \times (1000 + 1)$$

$$2342 \times 999 = 2342 \times (1000 - 1)$$

对二进制，也一样！

$$1010 \times 0111 = 1010 \times (1000 - 1)$$

$$1010 \times 1110 = 1010 \times (10000 - 10)$$

```
          0 0 1 1 0 1
×         0 0 1 1 1 0
-----------------------
        0 0 0 1 1 0 1
      0 0 1 1 0 1
    0 0 1 1 0 1
-----------------------
0 0 0 1 0 1 1 0 1 1 0
```

Booth乘法（补码乘法）

```
              0 0 1 1 0 1
        ×     0 0 1 1 1 0
  ---------------------------
        0 0 0 1 1 0 1
      0 0 1 1 0 1
    0 0 1 1 0 1
  0 0 0 1 0 1 1 0 1 1 0
```

<一略用心机需要3步

v.s

运用技巧只需2步一>

在此处减 $|X|$

在此处加 $|X|$

```
              0 0 1 1 0 1
        ×     0 0 1 1 1 0
  ---------------------------
    1 1 1 1 1 0 0 1 1
    0 0 1 1 0 1
  ---------------------------
    0 0 1 0 1 1 0 1 1 0
```

问题来了：如何高效地识别“一串1”的开始和结束?

Booth算法：

- 从右向左，遇到“1 0”就是“一串1”的开始，在1的位置减 $|X|$；
- 从右向左，遇到“0 1”就是“一串1”的结束，在0的位置加 $|X|$；
- 中间的“00”和“11”都不管！

y_i	y_{i-1}	操 作
0	0	部分积＋0，右移1位
0	1	部分积＋$[X]_{补}$，右移1
1	0	部分积＋$[-X]_{补}$，右移1
1	1	部分积＋0，右移1位

Booth乘法（补码乘法）

Booth算法：

- 从右向左，遇到“1 0”就是“一串1”的开始，在1的位置减$|X|$；
- 从右向左，遇到“0 1”就是“一串1”的结束，在0的位置加$|X|$；
- 中间的“00”和“11”都不管！

【例】X＝0.1010，Y＝－0.1101，利用Booth法求积。

【解】：

$[X]_{补}$ ＝00.1010

$[-X]_{补}$ ＝11.0110

$[Y]_{补}$ ＝11.0011

∴ **[X·Y]补 ＝ 1.01111110**

[Y]补含1位符号

-1位置尾数

符号		D				Y				Y_0	Y_{-1}	操作说明
0	0	0	0	0	0	1	0	0	1	1	0	D/Y初始化
1	1	0	1	1	0	1	0	0	1	1	0	尾数10，“串头”，-X
1	1	0	1	1	0	1	0	0	1	1	0	
1	1	1	0	1	1	0	1	0	0	1	1	尾数11，跳过
1	1	1	1	0	1	1	0	1	0	0	1	D/Y右移2位
0	0	1	0	1	0	0	0	1	0	0	1	尾数01，“串尾”，+X
0	0	0	1	1	1	1	0	1	0	0	1	
0	0	0	0	1	1	1	1	0	1	0	0	尾数00，跳过
0	0	0	0	0	1	1	1	1	0	1	0	D/Y右移2位
1	1	0	1	1	0	0	0	0	0	1	0	尾数10，“串头”，-X
1	1	0	1	1	1	1	1	1	0	1	0	
1	1	1.	0	1	1	1	1	1	1	0	1	D/Y右移2位

结束首位为符号

乘数用尽，计算结束

在此处减$|X|$

Booth乘法（补码乘法）

练习，[X]原=0.110110, [Y]原=1.101101，求积

解：[-X]补=1.001010;

[Y]补=1.010011

符号		D							Y						Y_0	Y_{-1}	操作说明
0	0	0	0	0	0	0	0		1	0	1	0	0	1	1	0	D/Y初始化
1	1	0	0	1	0	1	0		1	0	1	0	0	1	1	0	尾数10，“串头”，-X
1	1	0	0	1	0	1	0		1	0	1	0	0	1	1	0	
1	1	1	0	0	1	0	1	0	1	0	1	0	0		1	1	尾数11，跳过
1	1	1	1	0	0	1	0	1	0	1	0	1	0	0	0	1	D/Y右移2位
0	0	1	1	0	1	1	0	0	0	1	0	1	0	0	0	1	尾数01，“串尾”，+X
0	0	1	0	1	0	0	0	1	0	1	0	1	0	0	0	1	
0	0	0	1	0	1	0	0	0	1	0	1	0	1	0	0	0	尾数00，跳过
0	0	0	0	1	0	1	0	0	0	1	0	1	0	1	0	0	D/Y右移2位
1	1	0	0	1	0	1	0	0	0	0	0	1	0	1	0	0	尾数10，“串头”，-X
1	1	0	1	0	1	0	0	0	0	1	0	1	0	1	0	0	
1	1	1	0	1	0	1	0	0	0	0	1	0	1	0	1	1	D/Y右移2位
0	0	1	1	0	1	1	0	0	0	0	0	0	1	0	1	1	尾数10，“串头”，-X
0	0	1	0	0	0	0	0	0	0	0	1	0	1	0	1	1	
0	0	0	1	0	0	0	0	0	0	0	0	1	0	1	0	1	D/Y右移2位
1	1	0	0	1	0	1	0	0	0	0	0	0	0	1	0	1	尾数10，“串头”，-X
1	1	0	1	1	0	1	0	0	0	0	0	1	0	1	0	1	
1	1	1.	0	1	1	0	1	0	0	0	0	0	1	0	1	1	

乘数用尽，计算结束

∴ [X·Y]补 = 1.10110100000010

除法的“表格式计算”

关于除法的“表格式计算”

按理说，原码一位除法就是模拟手工除法:

- 符号$S_m = S_A \oplus S_b$, 数值$|m| = |A|/|B|$

例，[X]原=0.1011, [Y]原=1.1101，求商/余数

解： 符号，$S = 0 \oplus 1 = 1$； 数值部分如下：

模拟手工除法

```
                    0 .1 1 0 1
          ┌──────────────────
.1 1 0 1 ) .1 0 1 1
           0 0 0 0
           ─────────
           1 0 1 1 0
             1 1 0 1
             ─────────
           1 0 0 1 0
               1 1 0 1
               ─────────
             0 1 0 1 0
                 0 0 0 0
                 ───────
               1 0 1 0 0
                   1 1 0 1
                   ───────
.0 0 0 0 0 1 1 1
```

[X÷Y]原＝1.1001

余数＝1.1101×2-4

符号		余数（R）				商	操作说明
0	0	1	0	1	1		R初始化
0	0	0	0	0	0	0	不够减，商0
0	0	1	0	1	1		R=R - 0
0	1	0	1	1	0		R左移1位
1	1	0	0	1	1	1	够减，商1
0	0	1	0	0	1		R=R - D
0	1	0	0	1	0		R左移1位
1	1	0	0	1	1	1	够减，商1
0	0	0	1	0	1		R=R - D
0	0	1	0	1	0		R左移1位
0	0	0	0	0	0	0	不够减，商0
0	0	1	0	1	0		R=R - 0
0	1	0	1	0	0		R左移1位
1	1	0	0	1	1	1	够减，商1
0	0	0	1	1	1		R=R - D

关于除法的“表格式计算”

解决方案一：无脑直减，减多了原地恢复（恢复余数法）

例，[X]原=0.1011, [Y]原=1.1101，求商/余数

```
                    0 .1 1 0 1
.1 1 0 1 ) .1 0 1 1
           1 1 0 1
         - 0 0 1 0
         + 1 1 0 1
           1 0 1 1
           0 0 0 0
           1 0 1 1 0
             1 1 0 1
           1 0 0 1 0
             1 1 0 1
           0 1 0 1 0
               1 1 0 1
             - 0 0 1 1
             + 1 1 0 1
               1 0 1 0
               0 0 0 0
               1 0 1 0 0
                 1 1 0 1
           .0 0 0 0 0 1 1 1
```

恢复余数流程

使用临时变量R逐层累减除数D

层次	被除数(余数) R	商↓	说明
第一层 减1101	0 1 0 1 1 - 0 1 1 0 1 - 0 0 0 1 0		R > 0 无脑减D R = R - D
第二层 加1101	- 0 0 0 1 0 + 0 1 1 0 1 0 1 0 1 1		R<0 +D恢复R R = R + D
第三层 减0	0 1 0 1 1 - 0 0 0 0 0 0 1 0 1 1	0	0 < R < D 不够减、商0 R = R - D * 0
第四层 减1101	1 1 0 1 0 - 0 1 1 0 1 0 1 0 0 1	1	R << 1 够减，商1 R = R - D
第五层 减1101	1 0 0 1 0 - 0 1 1 0 1 0 0 1 0 1	1	R << 1 够减，商1 R = R - D
第六层 减1101	0 1 0 1 0 - 0 1 1 0 1 - 0 0 0 1 1		R > 0 无脑减D R = R - D
第七层 加1101	- 0 0 0 1 1 + 0 1 1 0 1 0 1 0 1 0		R<0 +D恢复R R = R + D
第八层 减0	0 1 0 1 0 - 0 0 0 0 0 0 1 0 1 0	0	0 < R < D 不够减、商0 R = R - D * 0
第九层 减1101	1 0 1 0 0 - 0 1 1 0 1 0 0 1 1 1	1	R << 1 够减，商1 R = R - D

关于除法的“表格式计算”

解决方案二：无脑直减，减多了原地不恢复（加减交替法）

例，[X]原=0.1011, [Y]原=1.1101，求商/余数

减成负值商0

```
                    0 .1 1 0 1
          ┌──────────────────────
.1 1 0 1 ) .1 0 1 1
           1 1 0 1
         ------------
         - 0 0 1 0 0
           + 1 1 0 1
         ------------
             1 0 0 1 0
               1 1 0 1
         ------------
                 1 0 1 0
                 1 1 0 1
         ------------
                   - 1 1 0
                 + 1 1 0 1
         ------------
         .0 0 0 0 0 1 1 1
```

符号	余数（R）	商	操作说明
0 0	1 0 1 1		R初始化
1 1	0 0 1 1		无脑直减，负了商0
1 1	1 1 1 0	0	
1 1	1 1 0 0		R左移1位
0 0	1 1 0 1		无脑直减，正了商1
0 0	1 0 0 1	1	
0 1	0 0 1 0		R左移1位
1 1	0 0 1 1	1	无脑直减，正了商1
0 0	0 1 0 1		
0 0	1 0 1 0		R左移1位
1 1	0 0 1 1		无脑直减，负了商0
1 1	1 1 0 1	0	
1 1	1 0 1 0		R左移1位
0 0	1 1 0 1	1	无脑直减，正了商1
0 0	0 1 1 1		

关于除法的“表格式计算”

例，[X]=-0.10001011, [Y]=0.1110，求商及余数，用恢复余数法

【解】

[X]原＝1.10001011

[Y]原＝0.1110

[－Y]补＝1.0010

商符＝1⊕0＝1

符号	余数（R）	商	操作说明
0 0	1 0 0 0 1 0 1 1		R初始化

关于除法的“表格式计算”

例，[X]=-0.10001011, [Y]=0.1110，求商及余数，用恢复余数法

【解】

[X]原＝1.10001011

[Y]原＝0.1110

[－Y]补＝1.0010

商符＝1⊕0＝1

[X÷Y]原＝1.1001

余数＝1.1101×2-4

符号	余数（R）	商	操作说明
0 0	1 0 0 0 1 0 1 1		R初始化
1 1	0 0 1 0		-X，负结果要恢复
1 1	1 0 1 0 1 0 1 1		
1 1	1 0 1 0 1 0 1 1		R不移动
0 0	1 1 1 0 0 0 0 0		+X，恢复R，商0
0 0	1 0 0 0 1 0 1 1	0	
0 1	0 0 0 1 0 1 1 0		R左移1位
1 1	0 0 1 0		-X，正了商0
0 0	0 0 1 1 0 1 1 0	1	
0 0	0 1 1 0 1 1 0 0		R左移1位
1 1	0 0 1 0		-X，负结果要恢复
1 1	1 0 0 0 1 1 0 0		
1 1	1 0 0 0 1 1 0 0		R左移1位
0 0	1 1 1 0 0 0 0 0		+X，恢复R，商0
0 0	0 1 1 0 1 1 0 0	0	
0 0	1 1 0 1 1 0 0 0		R左移1位
1 1	0 0 1 0		-X，负结果要恢复
1 1	1 1 1 1 1 0 0 0		
1 1	1 1 1 1 1 0 0 0		R左移1位
0 0	1 1 1 0		+X，恢复R，商0
0 0	1 1 0 1 1 0 0 0	0	
0 1	1 0 1 1 0 0 0 0		R左移1位
1 1	0 0 1 0		-X，正了商0
0 0	1 1 0 1	1	

关于除法的“表格式计算”

例，[X]=-0.10001011, [Y]=0.1110，求商及余数，用不恢复余数法

【解】

[X]原＝1.10001011

[Y]原＝0.1110

[－Y]补＝1.0010

商符＝1⊕0＝1

符号	余数（R）	商	操作说明
0 0	1 0 0 0 1 0 1 1		R初始化

关于除法的“表格式计算”

例，[X]=-0.10001011, [Y]=0.1110，求商及余数，用不恢复余数法

【解】

[X]原＝1.10001011

[Y]原＝0.1110

[－Y]补＝1.0010

商符＝1⊕0＝1

[X÷Y]原＝1.1001

余数＝1.1101×2-4

符号		余数（R）								商	操作说明
0	0	1	0	0	0	1	0	1	1		R初始化
1	1	0	0	1	0						-X，负了商0
1	1	1	0	1	0	1	0	1	1	0	
1	1	0	1	0	1	0	1	1	0		R左移1位
0	0	1	1	1	0	0	0	0	0		+X，正了商1
0	0	0	0	1	1	0	1	1	0	1	
0	0	0	1	1	0	1	1	0	0		R左移1位
1	1	0	0	1	0						-X，负了商0
1	1	1	0	0	0	1	1	0	0	0	
1	1	0	0	0	1	1	0	0	0		R左移1位
0	0	1	1	1	0						+，负了商0
1	1	1	1	1	1	1	0	0	0	0	
1	1	1	1	1	1	0	0	0	0		R左移1位
0	0	1	1	1	0	0	0	0	0		+X，正了商1
0	0	1	1	0	1	0	0	0	0	1	

关于补码除法

定点数运算—除法

回顾原码加减交替法

只有这个条件，和符号依赖！

当余数R<0时：

- **商0；**
- **恢复余数：R += D;**
- **余数左移：(R+D) << 1；**
- **进行第$i+1$轮，减D**
  - $(R+D) \ll 1 - D = 2R + D$
- **...**

这个过程，是符号无关的

定点数运算—除法

回顾原码加减交替法

当余数R<0时：

- 商0；
- 恢复余数：R += D;
- 余数左移：(R+D) << 1；
- 进行第$i+1$轮，减D
  - $(R+D) \ll 1 - D = 2R + D$
- ...

因此，我们得到补码交替除法

- 当余数R与D不同时：
  - 商0；
  - 恢复余数：R += D;
  - 余数左移：(R+D) << 1；
  - 进行第$i+1$轮，减D
    - $(R+D) \ll 1 - D = 2R + D$

定点数运算—除法

补码交替除法完整规则

1. R与D同号，-D
2. R与D异号，+D
3. 当新余数R与D相同时，商1
4. 当新余数R与D不同时，商0；
5. R左移1位，下一轮回到1
6. 除不尽时，商恒置1

此规则与原码交替除法完全兼容，表述上的超集

关于除法的“表格式计算”

例，例，[X]原=-0.1011, [Y]原=0.1101，求商/余数，用补码除法

【解】

[X]补＝1.0101

[Y]原＝0.1101

[－Y]补＝1.0011

符号	余数（R）	商	操作说明
1 1	0 1 0 1		R初始化

关于除法的“表格式计算”

例，例，[X]原=-0.1011, [Y]原=0.1101，求商/余数，用补码除法

【解】

[X]补＝1.0101

[Y]原＝0.1101

[－Y]补＝1.0011

1. R与D同号，-D
2. R与D异号，+D
3. 当新余数R与D相同时，商1
4. 当新余数R与D不同时，商0；
5. R左移1位，下一轮回到1
6. 除不尽时，商恒置1

符号		余数（R）				商	操作说明
1	1	0	1	0	1		R初始化
0	0	1	1	0	1		R与D异号，+D
0	0	0	0	1	0	1	R与D同号，商1
0	0	0	1	0	0		R左移1位
1	1	0	0	1	1		R与D同号，-D
1	1	0	1	1	1	0	R与D异号，商0
1	0	1	1	1	0		R左移1位
0	0	1	1	0	1		R与D异号，+D
1	1	1	0	1	1	0	R与D异号，商0
1	1	0	1	1	0		R左移1位
0	0	1	1	0	1		R与D异号，+D
0	0	0	0	1	1	1	R与D同号，商1
0	0	0	1	1	0		R左移1位
0	0	0	1	1	0	1	末尾恒1

[X÷Y]补＝1.0011

余数＝0.0110×2^{-4}

定点数运算—除法

解决方案三：补码交替加减法

例，[X]=-0.10001011, [Y]=0.1110，求商及余数

【解】

[X]补＝1.01110101

[Y]补＝0.1110

[－Y]补＝1.0010

1. R与D同号，-D
2. R与D异号，+D
3. 当新余数R与D相同时，商1
4. 当新余数R与D不同时，商0；
5. R左移1位，下一轮回到1
6. 除不尽时，商恒置1

符号	余数（R）	商	操作说明
1 1	0 1 1 1 0 1 0 1		R初始化

定点数运算—除法

解决方案三：补码交替加减法

例，[X]=-0.10001011, [Y]=0.1110，求商及余数

【解】

[X]补＝1.01110101

[Y]补＝0.1110

[－Y]补＝1.0010

1. R与D同号，-D
2. R与D异号，+D
3. 当新余数R与D相同时，商1
4. 当新余数R与D不同时，商0；
5. R左移1位，下一轮回到1
6. 除不尽时，商恒置1

[X÷Y]补＝1.01101

余数＝1.0011×2^{-4}

符号	余数（R）	商	操作说明
1 1	0 1 1 1 0 1 0 1		R初始化
0 0	1 1 1 0 0 0 0 0		R与D异号，+D
0 0	0 1 0 1 0 1 0 1	1	R与D同号，商1
0 0	1 0 1 0 1 0 1 0		R左移1位
1 1	0 0 1 0 0 0 0 0		R与D同号，-D
1 1	1 1 0 0 1 0 1 0	0	R与D异号，商0
1 1	1 0 0 1 0 1 0 0		R左移1位
0 0	1 1 1 0 0 0 0 0		R与D异号，+D
0 0	0 1 1 1 0 1 0 0	1	R与D同号，商1
0 0	1 1 1 0 1 0 0 0		R左移1位
1 1	0 0 1 0 0 0 0 0		R与D同号，-D
0 0	0 0 0 0 1 0 0 0	1	R与D同号，商1
0 0	0 0 0 1 0 0 0 0		R左移1位
1 1	0 0 1 0 0 0 0 0		R与D同号，-D
1 1	0 0 1 1 0 0 0 0	0	R与D异号，商0
1 1	0 0 1 1 0 0 0 0	1	末尾恒1